Formal Semantics

Concepts of Programming Languages

CAS CS 320

Outline

- Discuss **formal semantics** in general
- Look at **small-step** and **big-step** semantics with some examples

Introduction

A Thought Experiment

```bash
x=3
function f () {
    x=2
}
f
echo $x
```

Bash

```python
x = 3
def f():
    x = 2
f()
print(x)
```

Python

```ocaml
let x = 3
let f () =
  let x = 2 in
  ()
let _ = f ()
let _ = print_int x
```

OCaml

A Thought Experiment

```bash
x=3
function f () {
    x=2
}
f
echo $x
```

Bash

```python
x = 3
def f():
    x = 2
f()
print(x)
```

Python

```ocaml
let x = 3
let f () =
  let x = 2 in
  ()
let _ = f ()
let _ = print_int x
```

OCaml

Question. *How do we know what'll be printed?*

A Thought Experiment

```bash
x=3
function f () {
    x=2
}
f
echo $x
```

Bash

```python
x = 3
def f():
    x = 2
f()
print(x)
```

Python

```ocaml
let x = 3
let f () =
  let x = 2 in
  ()
let _ = f ()
let _ = print_int x
```

OCaml

Question. *How do we know what'll be printed?*

As programmers, we build intuitions by writing programs

A Thought Experiment

```bash
x=3
function f () {
    x=2
}
f
echo $x
```

Bash

```python
x = 3
def f():
    x = 2
f()
print(x)
```

Python

```ocaml
let x = 3
let f () =
  let x = 2 in
  ()
let _ = f ()
let _ = print_int x
```

OCaml

Question. *How do we know what'll be printed?*

As programmers, we build intuitions by writing programs

As PL designers, we have to *decide* this

Meaning

Meaning

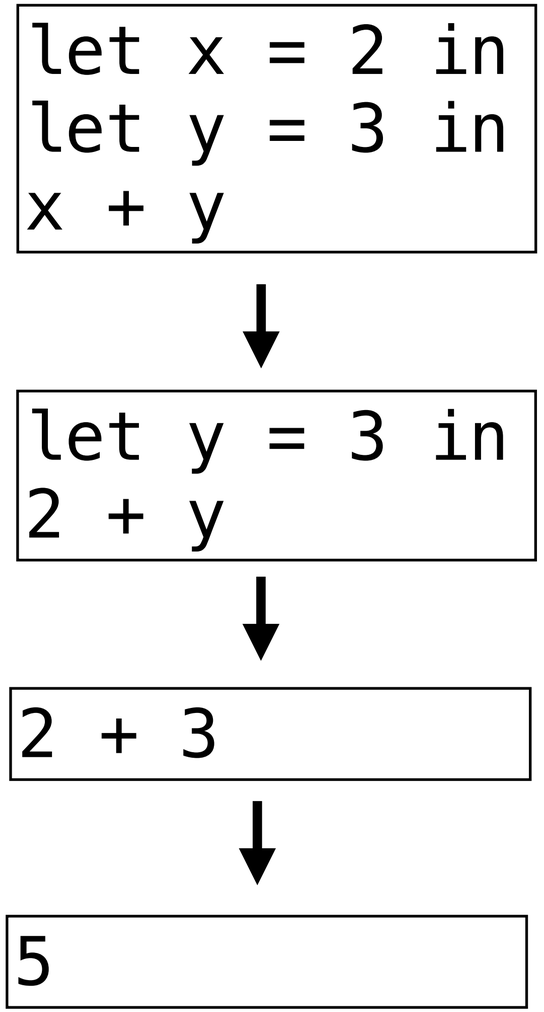
Syntax is interested in the *form* of a program

```
let x = 2 in
let y = 3 in
x + y
```

Meaning

Syntax is interested in the *form* of a program

Semantics is interested in the *meaning* of a program



Aside: Denotational vs. Operational Semantics

Aside: Denotational vs. Operational Semantics

Denotational semantics is interested in what a syntactic object "denotes" i.e. in interpreting programs as *mathematical objects*

$$1 + 2 * 3 + 4 = 11$$
$$1 + 12 - 2 = 11$$

Aside: Denotational vs. Operational Semantics

Denotational semantics is interested in what a syntactic object "denotes" i.e. in interpreting programs as *mathematical objects*

$$1 + 2 * 3 + 4 = 11$$
$$1 + 12 - 2 = 11$$

Operational semantics is interested in how a PL "operates" i.e. how a program *behaves* during execution

$$\begin{aligned} 1 + 2 * 3 + 4 &\longrightarrow 1 + 6 + 4 \\ &\longrightarrow 7 + 4 \\ &\longrightarrow 11 \end{aligned}$$

Aside: Denotational vs. Operational Semantics

Denotational semantics is interested in what a syntactic object "denotes" i.e. in interpreting programs as *mathematical objects*

$$1 + 2 * 3 + 4 = 11$$
$$1 + 12 - 2 = 11$$

This course

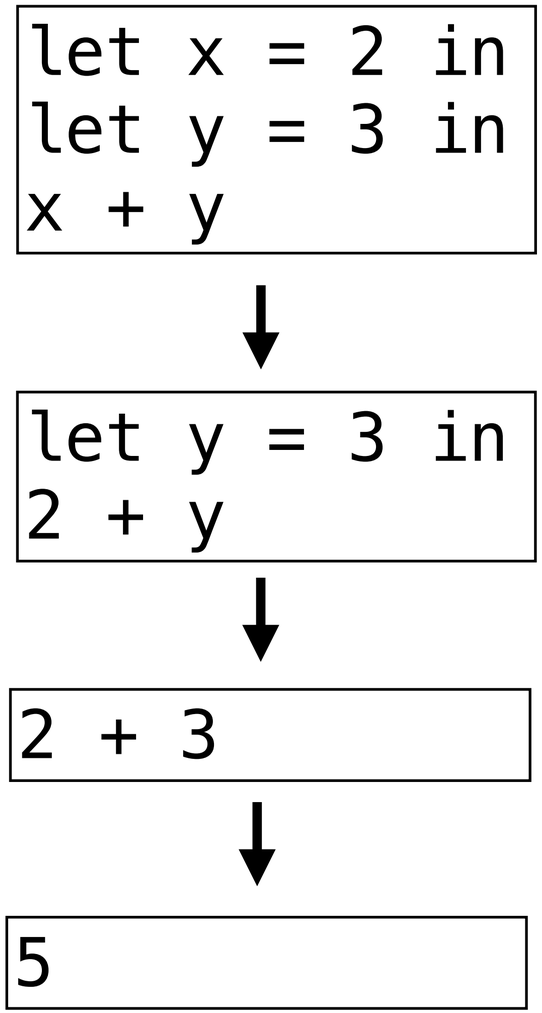
Operational semantics is interested in how a PL "operates" i.e. how a program *behaves* during execution

$$\begin{aligned} 1 + 2 * 3 + 4 &\longrightarrow 1 + 6 + 4 \\ &\longrightarrow 7 + 4 \\ &\longrightarrow 11 \end{aligned}$$

Small-Step vs. Big-Step Semantics

Small-Step vs. Big-Step Semantics

Small-step operational semantics is interested in *program transformation*, i.e., how a program transforms "one step at a time"



Small-Step vs. Big-Step Semantics

Small-step operational semantics is interested in *program transformation*, i.e., how a program transforms "one step at a time"

Big-step operational semantics is interested in *evaluation*, i.e., what is the value of the program once a program has finished evaluating

$$\dfrac{\dfrac{}{2 \Downarrow 2} \quad \dfrac{\dfrac{}{3 \Downarrow 3} \quad \dfrac{}{2 + 3 \Downarrow 5}}{\texttt{let y = 3 in 2 + y} \Downarrow 5}}{\textit{let x = 2 in let y = 3 in x + y} \Downarrow 5}$$

Small-Step vs. Big-Step Semantics

Small-step operational semantics is interested in *program transformation*, i.e., how a program transforms "one step at a time"

320Caml

Big-step operational semantics is interested in *evaluation*, i.e., what is the value of the program once a program has finished evaluating

$$\dfrac{\dfrac{}{2 \Downarrow 2} \quad \dfrac{\dfrac{}{3 \Downarrow 3} \quad \dfrac{}{2 + 3 \Downarrow 5}}{\texttt{let y = 3 in 2 + y} \Downarrow 5}}{\textit{let x = 2 in let y = 3 in x + y} \Downarrow 5}$$

Aside: Static vs. Dynamic Semantics

Aside: Static vs. Dynamic Semantics

Static semantics
refers to the meaning
given to a program
before it is evaluated

```
% ocaml silly.ml
File "./silly.ml", line 1, characters 8-9:
1 | let x = 2 +. 3.
            ^
Error: This expression has type int but an expression was
       expected of type
         float
  Hint: Did you mean `2.'?
```

Aside: Static vs. Dynamic Semantics

Static semantics
refers to the meaning given to a program *before* it is evaluated

Dynamic semantics
refers to the behavior of a program *during* evaluation

```
% ocaml silly.ml
File "./silly.ml", line 1, characters 8-9:
1 | let x = 2 +. 3.
            ^
Error: This expression has type int but an expression was
       expected of type
         float
  Hint: Did you mean `2.'?
```

```
utop # let x = 2 + 3;;
val x : int = 5
```

Aside: Static vs. Dynamic Semantics

Type checking

Static semantics
refers to the meaning given to a program *before* it is evaluated

```
% ocaml silly.ml
File "./silly.ml", line 1, characters 8-9:
1 | let x = 2 +. 3.
            ^
Error: This expression has type int but an expression was
       expected of type
         float
  Hint: Did you mean `2.'?
```

Dynamic semantics
refers to the behavior of a program *during* evaluation

```
utop # let x = 2 + 3;;
val x : int = 5
```

Aside: Static vs. Dynamic Semantics

Static semantics
refers to the meaning
given to a program
before it is evaluated

Type checking

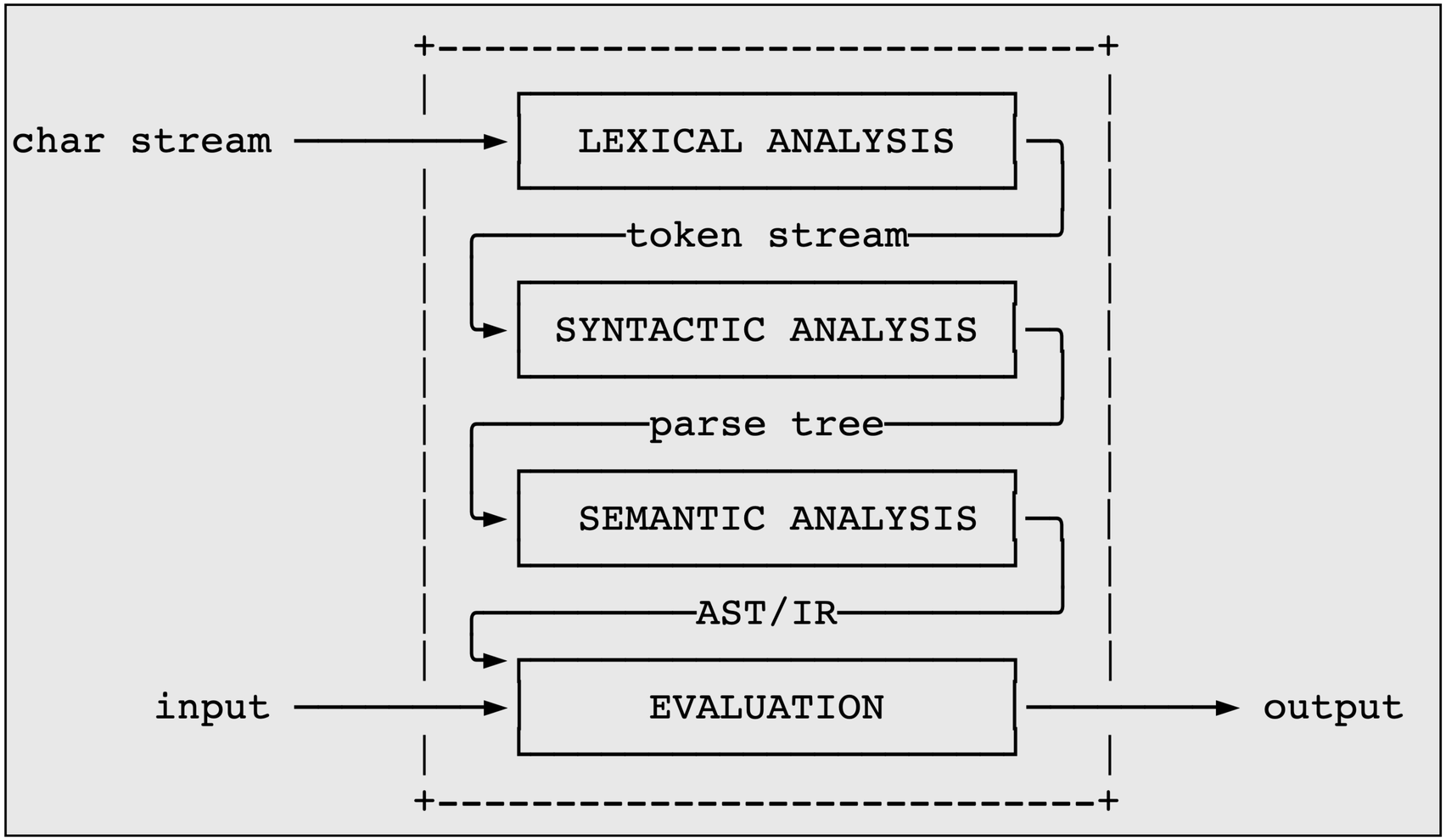
```
% ocaml silly.ml
File "./silly.ml", line 1, characters 8-9:
1 | let x = 2 +. 3.
            ^
Error: This expression has type int but an expression was
       expected of type
         float
  Hint: Did you mean `2.'?
```

Dynamic semantics
refers to the behavior
of a program *during*
evaluation

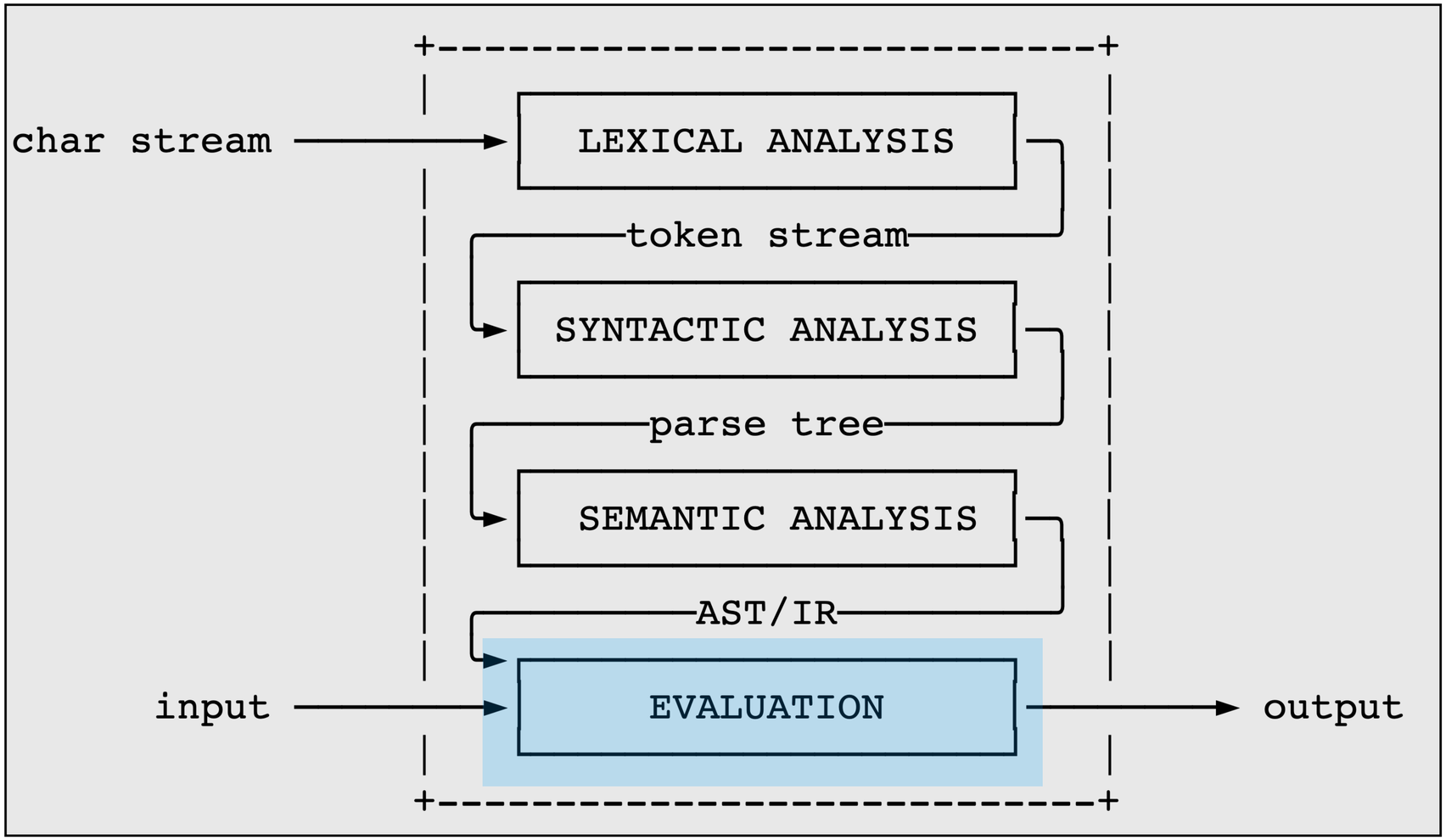
Evaluation

```
utop # let x = 2 + 3;;
val x : int = 5
```

Recall: The Picture

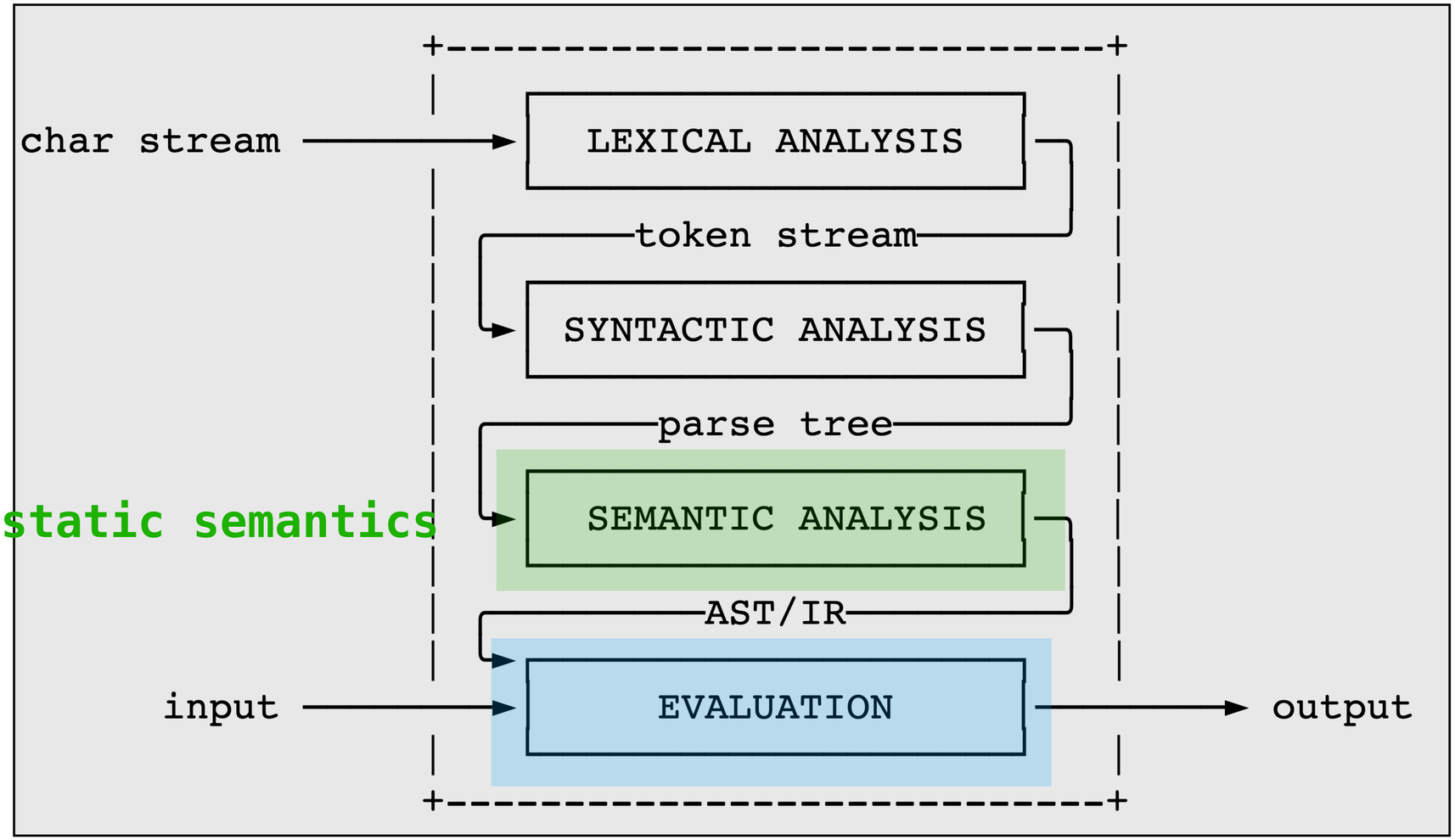


Recall: The Picture



dynamic semantics (this week)

Recall: The Picture



dynamic semantics (this week)

Small-Step Operational Semantics

Small-Step Semantic Judgment

$$(S, p) \longrightarrow (S', p')$$

Small-Step Semantic Judgment

$$(S, p) \longrightarrow (S', p')$$

Small-step semantics formalizes a **"step by step"** computation that reduces a syntactic object

Small-Step Semantic Judgment

$$(S, p) \longrightarrow (S', p')$$

Small-step semantics formalizes a **"step by step"** computation that reduces a syntactic object

Notation. We write $e \longrightarrow e'$ to mean e reduces to e' in a single step

Small-Step Semantic Judgment

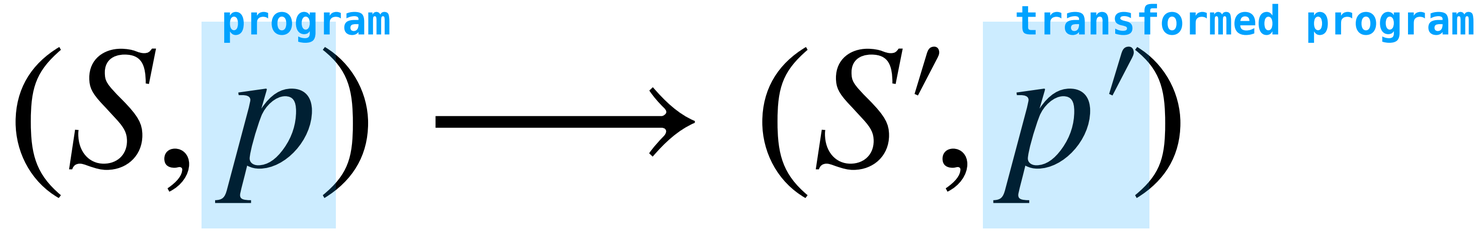
$$(S, p) \longrightarrow (S', p')$$

Small-step semantics formalizes a **"step by step"** computation that reduces a syntactic object

Notation. We write $e \longrightarrow e'$ to mean e reduces to e' in a single step

We define small-step semantics on a **configuration,** which is a program together with some stateful information

Small-Step Semantic Judgment

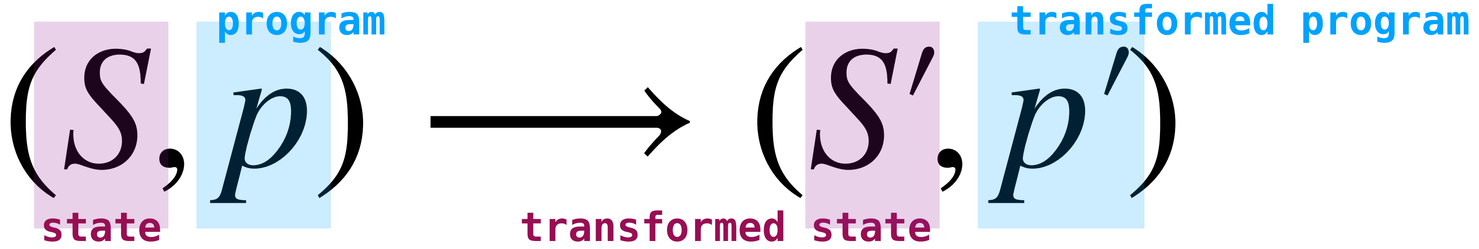


Small-step semantics formalizes a **"step by step"** computation that reduces a syntactic object

Notation. We write $e \longrightarrow e'$ to mean e reduces to e' in a single step

We define small-step semantics on a **configuration,** which is a program together with some stateful information

Small-Step Semantic Judgment

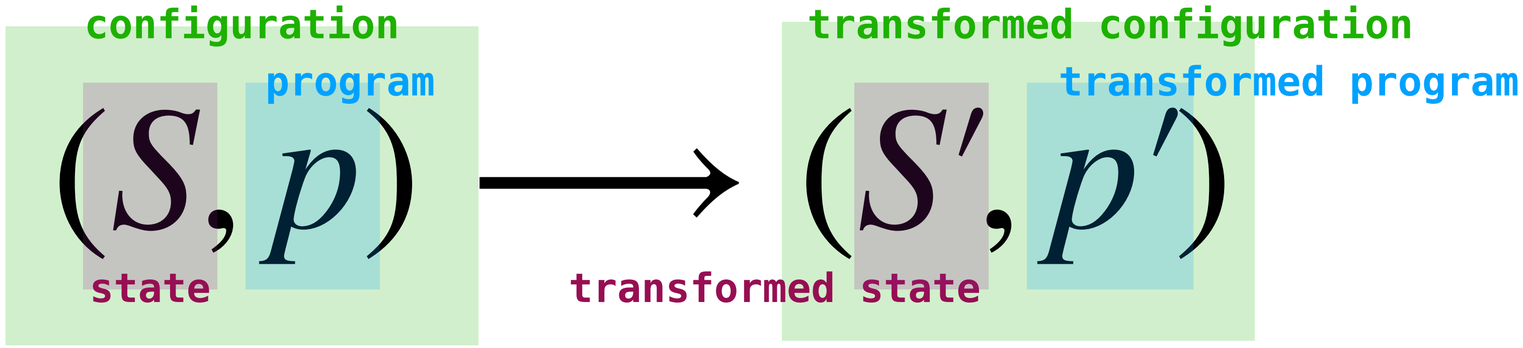


Small-step semantics formalizes a **"step by step"** computation that reduces a syntactic object

Notation. We write $e \longrightarrow e'$ to mean e reduces to e' in a single step

We define small-step semantics on a **configuration,** which is a program together with some stateful information

Small-Step Semantic Judgment

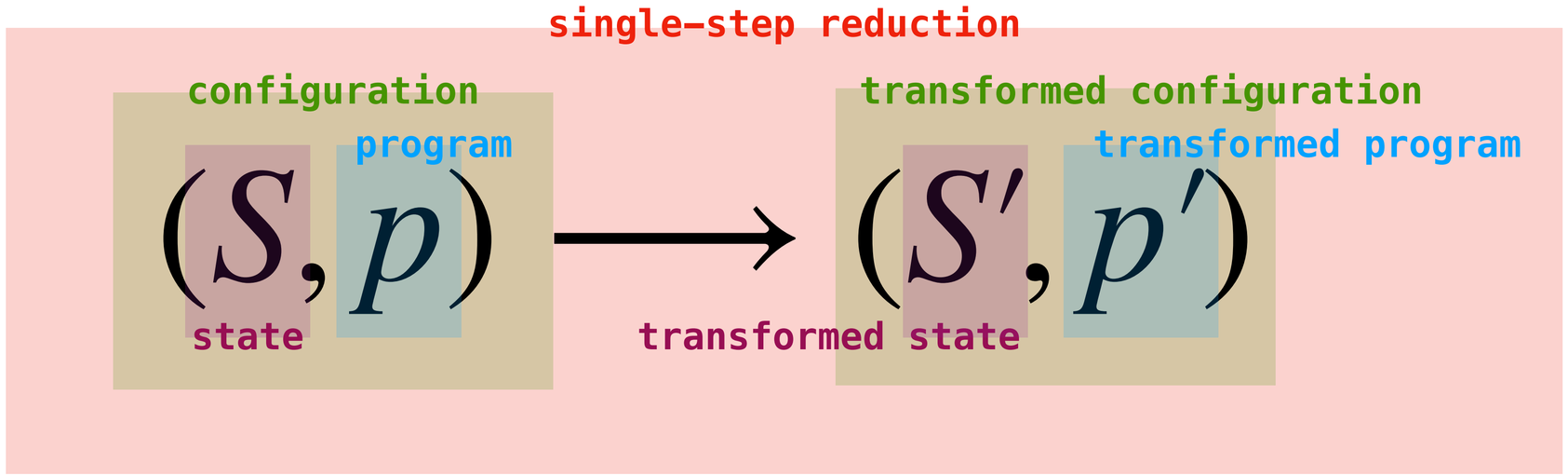


Small-step semantics formalizes a **"step by step"** computation that reduces a syntactic object

Notation. We write $e \longrightarrow e'$ to mean e reduces to e' in a single step

We define small-step semantics on a **configuration,** which is a program together with some stateful information

Small-Step Semantic Judgment

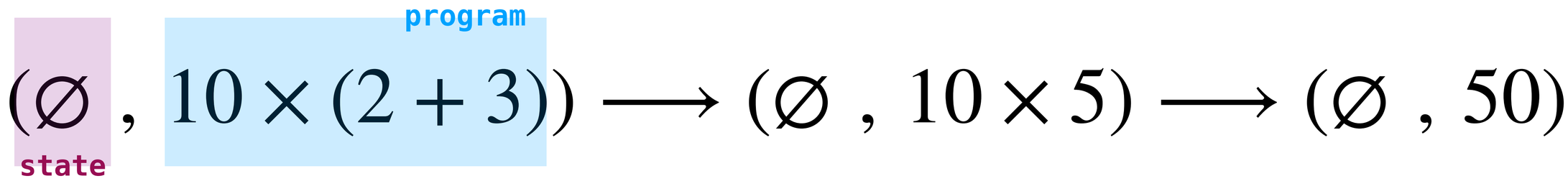


Small-step semantics formalizes a **"step by step"** computation that reduces a syntactic object

Notation. We write $e \longrightarrow e'$ to mean e reduces to e' in a single step

We define small-step semantics on a **configuration,** which is a program together with some stateful information

Example: Arithmetic Expressions



State: none

Program: Arithmetic expression

Example: OCaml

$$(\varnothing\ ,\ \texttt{let x = 3 in if x > 10 then 4 else 5}) \longrightarrow (\varnothing\ ,\ \texttt{if 3 > 10 then 4 else 5}) \longrightarrow (\varnothing\ ,\ \texttt{if false then 4 else 5}) \longrightarrow (\varnothing\ ,\ 5)$$

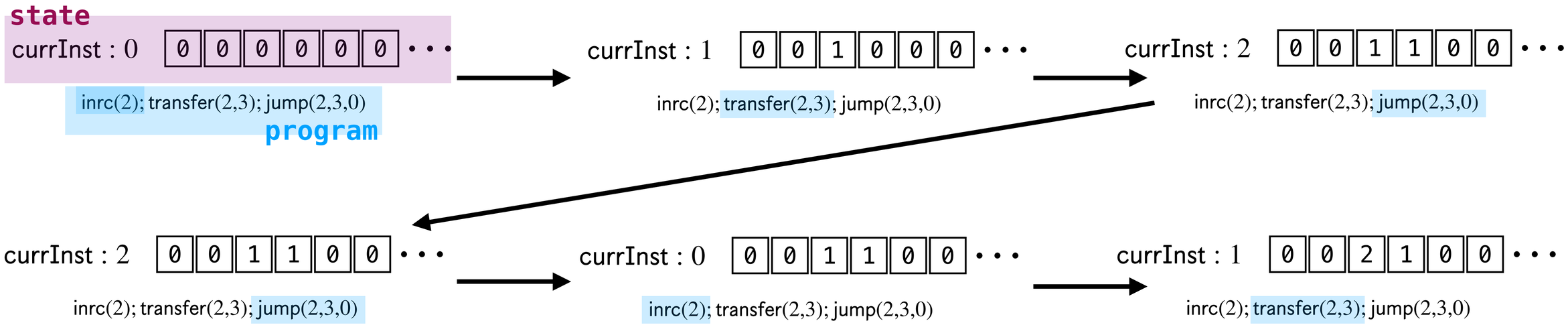
state — program

State: None

Program: OCaml expression

For purely functional languages
there is no state

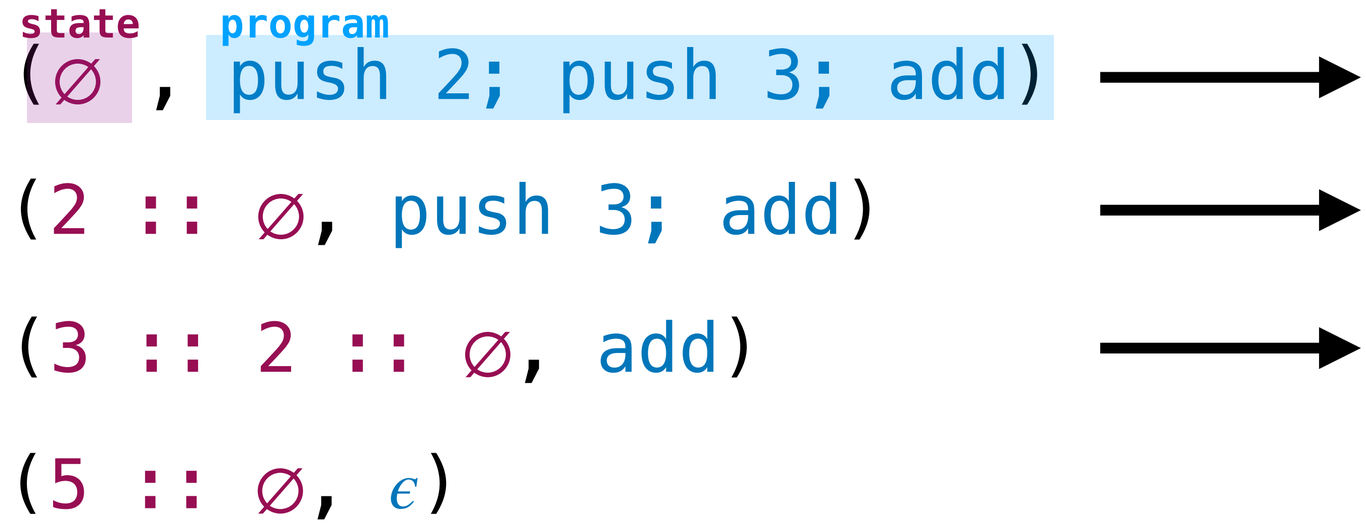
Example: Register Machines



State: (current instruction pointer) +
(collection of number registers)

Program: Sequence of commands

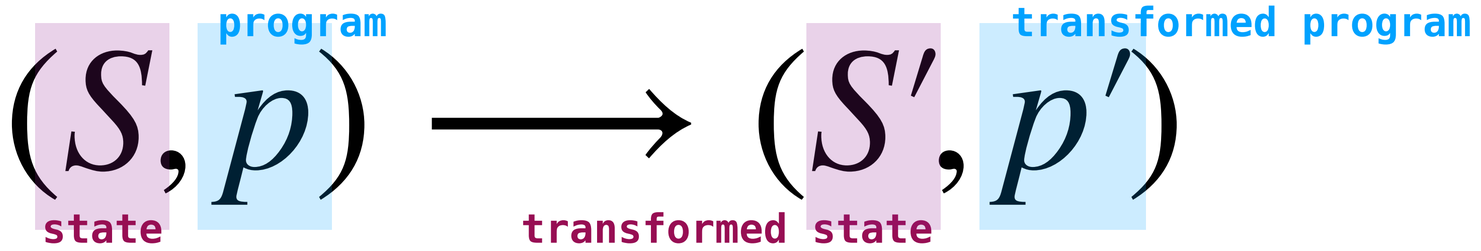
Example: Stack Language



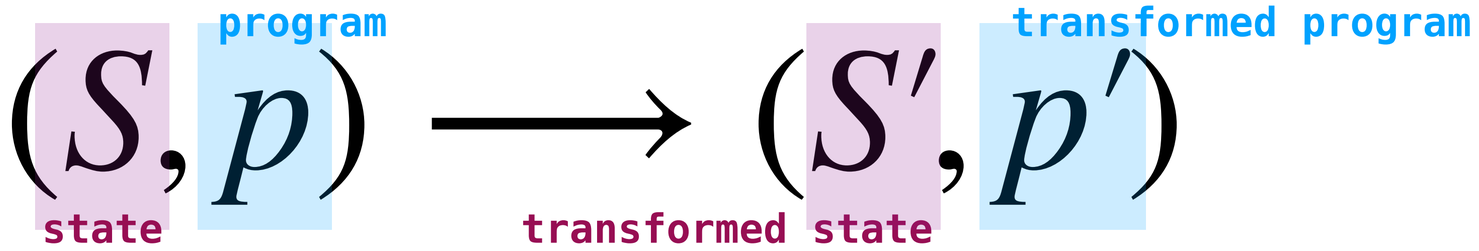
State: stack (i.e., list) of values

Program: sequence of commands for manipulating the stack

Small-Step Semantics

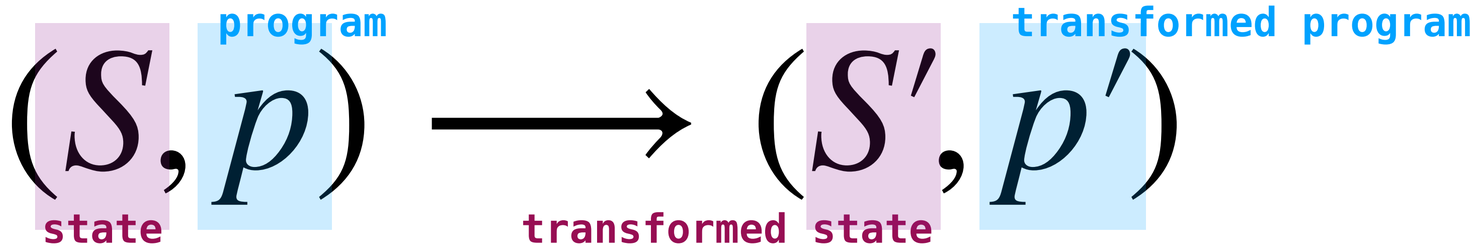


Small-Step Semantics



Defining a small-step semantics means three things:

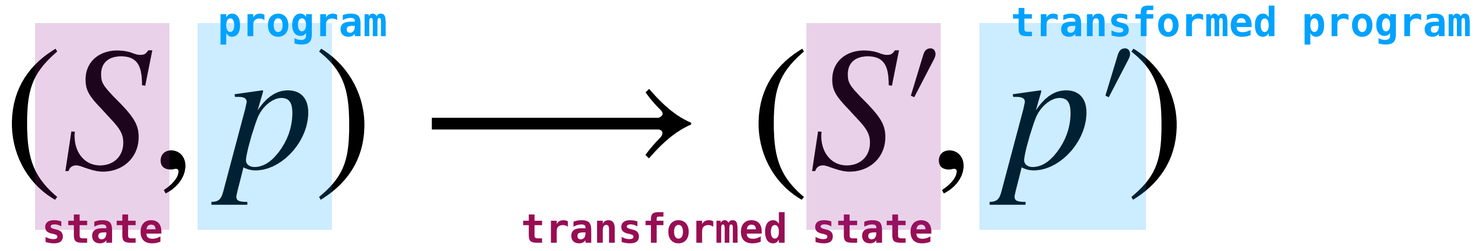
Small-Step Semantics



Defining a small-step semantics means three things:

» What kind of **state**?

Small-Step Semantics

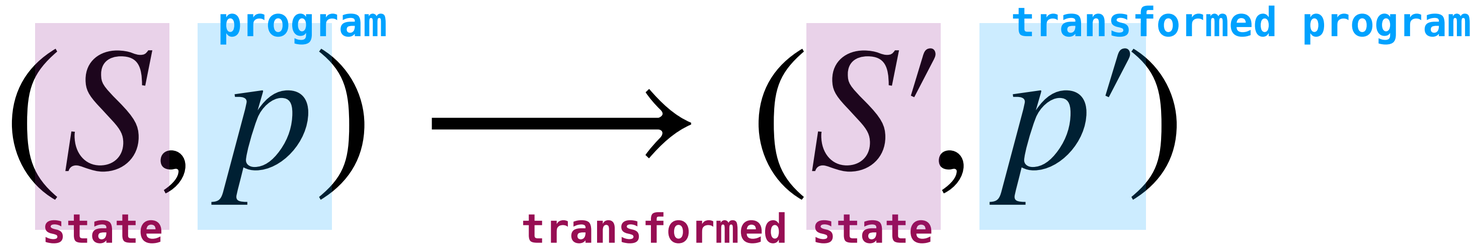


Defining a small-step semantics means three things:

» What kind of **state**?

» What **rules** for transforming configurations?

Small-Step Semantics



Defining a small-step semantics means three things:

- What kind of **state**?
- What **rules** for transforming configurations?
- What are the **successful** configurations?

Example

```
<expr> ::= ( <op> <expr> <expr> )
         | <bool> | <int>
<op>   ::= add | sub | eq
<bool> ::= true | false
<int>  ::= ...
```

Example

```
<expr> ::= ( <op> <expr> <expr> )
         | <bool> | <int>
<op>   ::= add | sub | eq
<bool> ::= true | false
<int>  ::= ...
```

$$\frac{e_1 \longrightarrow e_1'}{(\texttt{add}\ e_1\ e_2) \longrightarrow (\texttt{add}\ e_1'\ e_2)}\ \texttt{add-left}$$

$$\frac{e_2 \longrightarrow e_2'}{(\texttt{add}\ e_1\ e_2) \longrightarrow (\texttt{add}\ e_1\ e_2')}\ \texttt{add-right}$$

$$\frac{n_1\ \texttt{is a number} \qquad n_2\ \texttt{is a number}}{(\texttt{add}\ n_1\ n_2) \longrightarrow n_1 + n_2}\ \texttt{add-ok}$$

Example

```
<expr> ::= ( <op> <expr> <expr> )
         | <bool> | <int>
<op>   ::= add | sub | eq
<bool> ::= true | false
<int>  ::= ...
```

$$\frac{e_1 \longrightarrow e_1'}{(\texttt{add}\ e_1\ e_2) \longrightarrow (\texttt{add}\ e_1'\ e_2)}\ \texttt{add-left}$$

$$\frac{e_2 \longrightarrow e_2'}{(\texttt{add}\ e_1\ e_2) \longrightarrow (\texttt{add}\ e_1\ e_2')}\ \texttt{add-right}$$

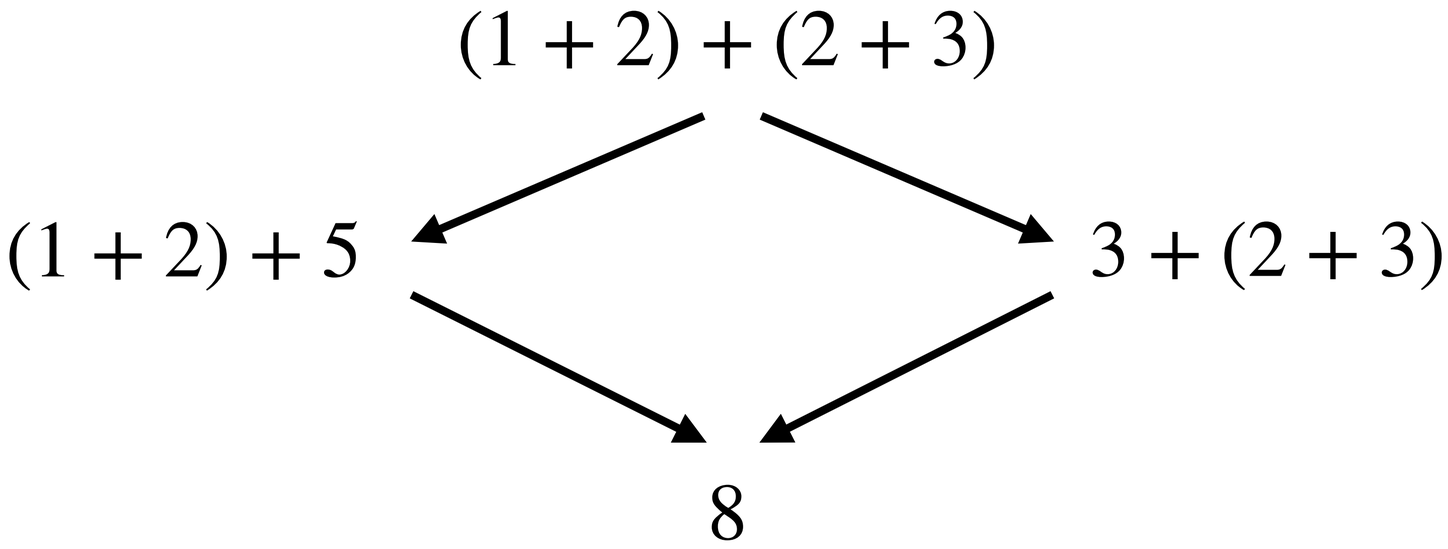
$$\frac{n_1\ \texttt{is a number} \qquad n_2\ \texttt{is a number}}{(\texttt{add}\ n_1\ n_2) \longrightarrow n_1 + n_2}\ \texttt{add-ok}$$

$$\frac{e_1 \longrightarrow e_1'}{(\texttt{sub}\ e_1\ e_2) \longrightarrow (\texttt{sub}\ e_1'\ e_2)}\ \texttt{sub-left}$$

$$\frac{e_2 \longrightarrow e_2'}{(\texttt{sub}\ e_1\ e_2) \longrightarrow (\texttt{sub}\ e_1\ e_2')}\ \texttt{sub-right}$$

$$\frac{n_1\ \texttt{is a number} \qquad n_2\ \texttt{is a number}}{(\texttt{sub}\ n_1\ n_2) \longrightarrow n_1 - n_2}\ \texttt{sub-ok}$$

Nondeterminism



There may be multiple choices of reductions

We should design our rules to *avoid* this

Nondeterminism

$$\frac{\texttt{add}\ 1\ 2 \longrightarrow 3}{(\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3)) \longrightarrow (\texttt{add}\ 3\ (\texttt{add}\ 2\ 3))}\ \texttt{add-left}$$

$$\frac{\texttt{add}\ 2\ 3 \longrightarrow 5}{(\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3)) \longrightarrow (\texttt{add}\ (\texttt{add}\ 1\ 2)\ 5)}\ \texttt{add-right}$$

Nondeterminism

$$\frac{\texttt{add}\ 1\ 2 \longrightarrow 3}{(\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3)) \longrightarrow (\texttt{add}\ 3\ (\texttt{add}\ 2\ 3))}\ \texttt{add-left}$$

$$\frac{\texttt{add}\ 2\ 3 \longrightarrow 5}{(\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3)) \longrightarrow (\texttt{add}\ (\texttt{add}\ 1\ 2)\ 5)}\ \texttt{add-right}$$

There are two reductions from (add (add 1 2) (add 2 3)) in our current rule set

Nondeterminism

$$\frac{\texttt{add}\ 1\ 2 \longrightarrow 3}{(\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3)) \longrightarrow (\texttt{add}\ 3\ (\texttt{add}\ 2\ 3))}\ \texttt{add-left}$$

$$\frac{\texttt{add}\ 2\ 3 \longrightarrow 5}{(\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3)) \longrightarrow (\texttt{add}\ (\texttt{add}\ 1\ 2)\ 5)}\ \texttt{add-right}$$

There are two reductions from (add (add 1 2) (add 2 3)) in our current rule set

We can avoid this by *breaking symmetry*

Example: Addition

```
<expr> ::= ( <op> <expr> <expr> )
         | <bool> | <int>
<op>   ::= add | sub | eq
<bool> ::= true | false
<int>  ::= ...
```

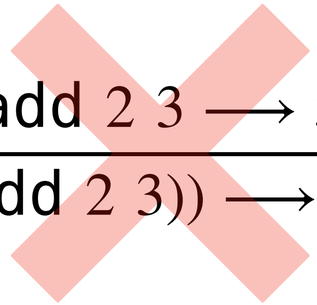
$$\frac{e_1 \longrightarrow e_1'}{(\texttt{add}\ e_1\ e_2) \longrightarrow (\texttt{add}\ e_1'\ e_2)}\ \texttt{add-left}$$

$$\frac{v\ \texttt{is a number} \qquad e_2 \longrightarrow e_2'}{(\texttt{add}\ v\ e_2) \longrightarrow (\texttt{add}\ v\ e_2')}\ \texttt{add-right}$$

$$\frac{n_1\ \texttt{is a number} \qquad n_2\ \texttt{is a number}}{(\texttt{add}\ n_1\ n_2) \longrightarrow n_1 + n_2}\ \texttt{add-ok}$$

Enforcing an Evaluation Strategy

$$\frac{\texttt{add}\ 1\ 2 \longrightarrow 3}{(\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3)) \longrightarrow (\texttt{add}\ 3\ (\texttt{add}\ 2\ 3))}\ \texttt{add-left}$$



$$\frac{\texttt{add}\ 2\ 3 \longrightarrow 5}{(\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3)) \longrightarrow (\texttt{add}\ (\texttt{add}\ 1\ 2)\ 5)}\ \texttt{add-right}$$

The new rule enforces that arguments of **add** are evaluated from left to right

Example

```
<expr> ::= ( <op> <expr> <expr> )
         | <bool> | <int>
<op>   ::= add | sub | eq
<bool> ::= true | false
<int>  ::= ...
```

Example

```
<expr> ::= ( <op> <expr> <expr> )
         | <bool> | <int>
<op>   ::= add | sub | eq
<bool> ::= true | false
<int>  ::= ...
```

$$\frac{e_1 \longrightarrow e_1'}{(\texttt{add}\ e_1\ e_2) \longrightarrow (\texttt{add}\ e_1'\ e_2)}\ \texttt{add-left}$$

$$\frac{n\ \texttt{is a number} \qquad e_2 \longrightarrow e_2'}{(\texttt{add}\ n\ e_2) \longrightarrow (\texttt{add}\ n\ e_2')}\ \texttt{add-right}$$

$$\frac{n_1\ \texttt{is a number} \qquad n_2\ \texttt{is a number}}{(\texttt{add}\ n_1\ n_2) \longrightarrow n_1 + n_2}\ \texttt{add-ok}$$

Example

```
<expr> ::= ( <op> <expr> <expr> )
         | <bool> | <int>
<op>   ::= add | sub | eq
<bool> ::= true | false
<int>  ::= ...
```

$$\frac{e_1 \longrightarrow e_1'}{(\texttt{add}\ e_1\ e_2) \longrightarrow (\texttt{add}\ e_1'\ e_2)}\ \texttt{add-left}$$

$$\frac{n\ \texttt{is a number} \qquad e_2 \longrightarrow e_2'}{(\texttt{add}\ n\ e_2) \longrightarrow (\texttt{add}\ n\ e_2')}\ \texttt{add-right}$$

$$\frac{n_1\ \texttt{is a number} \qquad n_2\ \texttt{is a number}}{(\texttt{add}\ n_1\ n_2) \longrightarrow n_1 + n_2}\ \texttt{add-ok}$$

$$\frac{e_1 \longrightarrow e_1'}{(\texttt{sub}\ e_1\ e_2) \longrightarrow (\texttt{sub}\ e_1'\ e_2)}\ \texttt{sub-left}$$

$$\frac{n\ \texttt{is a number} \qquad e_2 \longrightarrow e_2'}{(\texttt{sub}\ n\ e_2) \longrightarrow (\texttt{sub}\ n\ e_2')}\ \texttt{sub-right}$$

$$\frac{n_1\ \texttt{is a number} \qquad n_2\ \texttt{is a number}}{(\texttt{sub}\ n_1\ n_2) \longrightarrow n_1 - n_2}\ \texttt{sub-ok}$$

Practice Problem

```
<expr> ::= ( <op> <expr> <expr> )
         | <bool> | <int>
<op>   ::= add | sub | eq
<bool> ::= true | false
<int>  ::= ...
```

*Write down the reduction rules for **eq** (to the best of your ability) so that the left argument is evaluated before the right argument*

Answer

```
<expr> ::= ( <op> <expr> <expr> )
         | <bool> | <int>
<op>   ::= add | sub | eq
<bool> ::= true | false
<int>  ::= ...
```

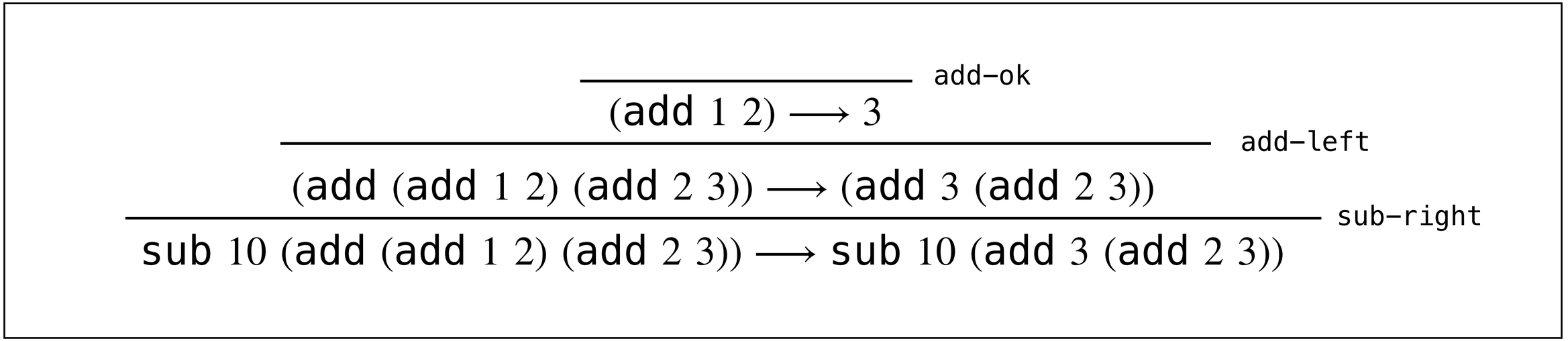
Two Questions

Given a small-step semantics, we can ask:

» Show that $C \longrightarrow C'$

» Given C, determine a configuration C' such that $C \longrightarrow C'$ (and show that it holds)

Recall: Derivations

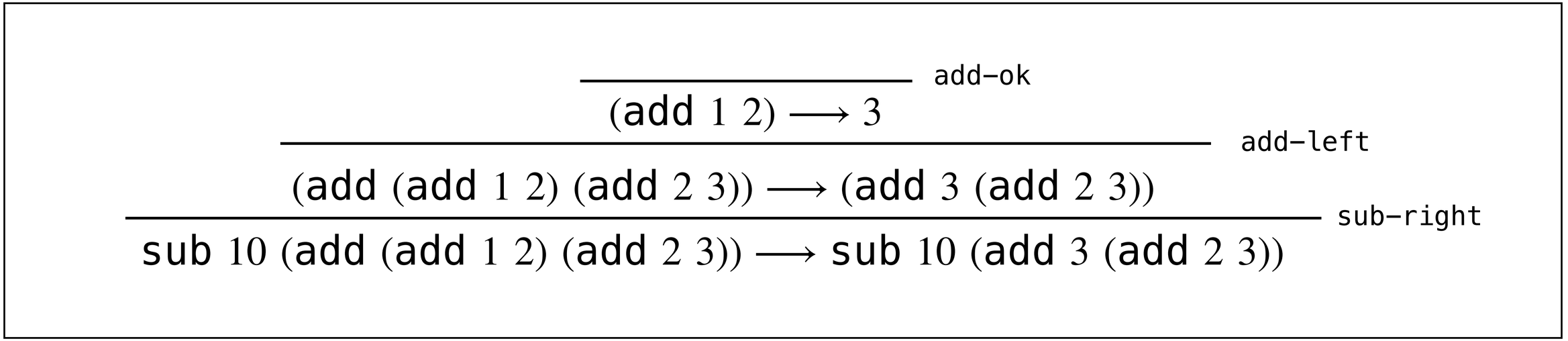


Recall: Derivations

$$\dfrac{\dfrac{\dfrac{}{(\text{add } 1\ 2) \longrightarrow 3}\ \texttt{add-ok}}{(\text{add } (\text{add } 1\ 2)\ (\text{add } 2\ 3)) \longrightarrow (\text{add } 3\ (\text{add } 2\ 3))}\ \texttt{add-left}}{\text{sub } 10\ (\text{add } (\text{add } 1\ 2)\ (\text{add } 2\ 3)) \longrightarrow \text{sub } 10\ (\text{add } 3\ (\text{add } 2\ 3))}\ \texttt{sub-right}$$

A **derivation** is a tree of reductions, gotten by applying reduction rules

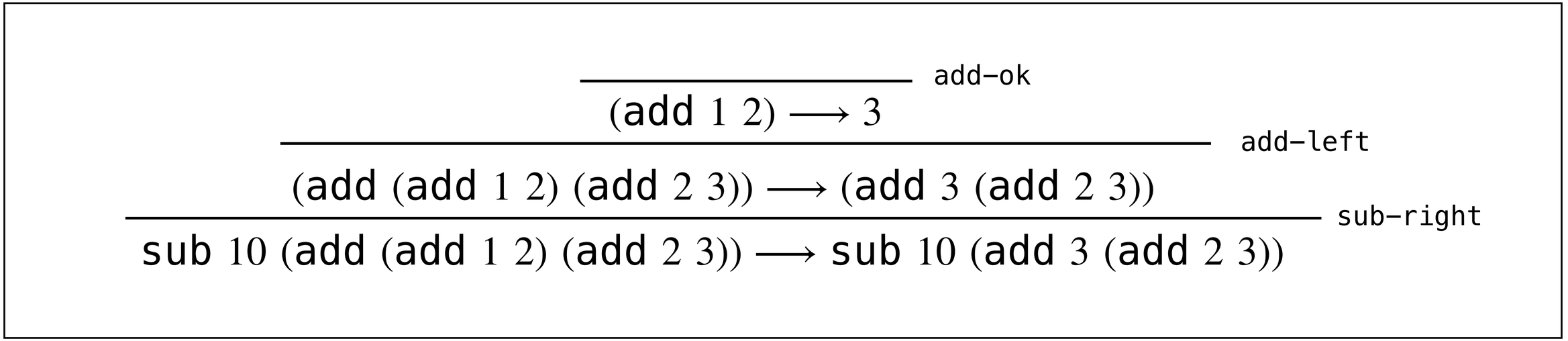
Recall: Derivations



A **derivation** is a tree of reductions, gotten by applying reduction rules

A derivation is a proof that the reduction step is valid in the operational semantics

Recall: Derivations



A **derivation** is a tree of reductions, gotten by applying reduction rules

A derivation is a proof that the reduction step is valid in the operational semantics

We've done this

Two Questions

Given a small-step semantics, we can ask:

» Show that $C \longrightarrow C'$

» Given C, determine a configuration C' such that $C \longrightarrow C'$ (and show that it holds)

Single-Step Evaluation

$$(\texttt{sub}\ 10\ (\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3))) \longrightarrow ???$$

Single-Step Evaluation

$$(\texttt{sub}\ 10\ (\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3))) \longrightarrow ???$$

The more "realistic" situation is to *figure out* what a program evaluates to in a single step

Example

$$\frac{e_1 \longrightarrow e_1'}{(\texttt{add}\ e_1\ e_2) \longrightarrow (\texttt{add}\ e_1'\ e_2)}\ \texttt{add-left}$$

$$\frac{n\ \texttt{is a number} \qquad e_2 \longrightarrow e_2'}{(\texttt{add}\ n\ e_2) \longrightarrow (\texttt{add}\ n\ e_2')}\ \texttt{add-right}$$

$$\frac{n_1\ \texttt{is a number} \qquad n_2\ \texttt{is a number}}{(\texttt{add}\ n_1\ n_2) \longrightarrow n_1 + n_2}\ \texttt{add-ok}$$

$$(\texttt{sub}\ 10\ (\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3))) \longrightarrow ???$$

Practice Problem

$$\frac{e_1 \longrightarrow e_1'}{(\texttt{add}\ e_1\ e_2) \longrightarrow (\texttt{add}\ e_1'\ e_2)}\ \texttt{add-left} \qquad \frac{e_2 \longrightarrow e_2'}{(\texttt{add}\ e_1\ e_2) \longrightarrow (\texttt{add}\ e_1\ e_2')}\ \texttt{add-right}$$

$$\frac{n_1\ \texttt{is a number} \quad n_2\ \texttt{is a number}}{(\texttt{add}\ n_1\ n_2) \longrightarrow n_1 + n_2}\ \texttt{add-ok}$$

$$\frac{e_1 \longrightarrow e_1'}{(\texttt{sub}\ e_1\ e_2) \longrightarrow (\texttt{sub}\ e_1'\ e_2)}\ \texttt{sub-left} \qquad \frac{e_2 \longrightarrow e_2'}{(\texttt{sub}\ e_1\ e_2) \longrightarrow (\texttt{sub}\ e_1\ e_2')}\ \texttt{sub-right}$$

$$\frac{n_1\ \texttt{is a number} \quad n_2\ \texttt{is a number}}{(\texttt{sub}\ n_1\ n_2) \longrightarrow n_1 - n_2}\ \texttt{sub-ok}$$

$$(\texttt{sub}\ 10\ (\texttt{add}\ 3\ (\texttt{add}\ 2\ 3))) \longrightarrow (\texttt{sub}\ 10\ (\texttt{add}\ 3\ 5))$$

Give a derivation of the above reduction

Answer

$$(\texttt{sub}\ 10\ (\texttt{add}\ 3\ (\texttt{add}\ 2\ 3))) \longrightarrow (\texttt{sub}\ 10\ (\texttt{add}\ 3\ 5))$$

Multi-Step Reduction Relation

$$\frac{}{C \longrightarrow^\star C}\ \text{refl} \qquad \frac{C \longrightarrow C' \qquad C' \longrightarrow^\star D}{C \longrightarrow^\star D}\ \text{step}$$

Given any single-step reduction relation, we have a corresponding **multi-step reduction relation:**

» Every configuration reduces to itself **(reflexivity)**

» Every $\longrightarrow^\star$ reduction can be extended by a single step **(transitivity)**

Two Questions (Again)

Once we have an operational semantics, there are two questions we can ask (as PL designers and on the final exam):

» Show that $C \longrightarrow^\star C'$

» Given C, determine a configuration C' such that $C \longrightarrow^\star C'$ **and** C' **cannot be reduced**

Two Questions (Again)

Once we have an operational semantics, there are two questions we can ask (as PL designers and on the final exam):

» Show that $C \longrightarrow^\star C'$

» Given C, determine a configuration C' such that $C \longrightarrow^\star C'$ **and** C' **cannot be reduced**

How To: Derivations of Multi-Step Reductions

$$\texttt{sub}\ 10\ (\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3)) \longrightarrow^{\star} 2$$

want to show

» Derive all necessary single-step evaluations

How To: Derivations of Multi-Step Reductions

$$\texttt{sub}\ 10\ (\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3)) \longrightarrow^{\star} 2$$

want to show

$$\texttt{sub}\ 10\ (\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3)) \longrightarrow \texttt{sub}\ 10\ (\texttt{add}\ 3\ (\texttt{add}\ 2\ 3))$$ (we did this)

» Derive all necessary single-step evaluations

How To: Derivations of Multi-Step Reductions

$$\texttt{sub}\ 10\ (\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3)) \longrightarrow^{\star} 2$$

want to show

$\texttt{sub}\ 10\ (\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3)) \longrightarrow \texttt{sub}\ 10\ (\texttt{add}\ 3\ (\texttt{add}\ 2\ 3))$ (we did this)

$\texttt{sub}\ 10\ (\texttt{add}\ 3\ (\texttt{add}\ 2\ 3)) \longrightarrow \texttt{sub}\ 10\ (\texttt{add}\ 3\ 5)$ (you did this)

» Derive all necessary single-step evaluations

How To: Derivations of Multi-Step Reductions

$$\texttt{sub}\ 10\ (\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3)) \longrightarrow^{\star} 2$$

want to show

$\texttt{sub}\ 10\ (\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3)) \longrightarrow \texttt{sub}\ 10\ (\texttt{add}\ 3\ (\texttt{add}\ 2\ 3))$ (we did this)

$\texttt{sub}\ 10\ (\texttt{add}\ 3\ (\texttt{add}\ 2\ 3)) \longrightarrow \texttt{sub}\ 10\ (\texttt{add}\ 3\ 5)$ (you did this)

$\texttt{sub}\ 10\ (\texttt{add}\ 3\ 5) \longrightarrow \texttt{sub}\ 10\ 8$ (exercise)

» Derive all necessary single-step evaluations

How To: Derivations of Multi-Step Reductions

$$\texttt{sub}\ 10\ (\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3)) \longrightarrow^{\star} 2$$

want to show

$\texttt{sub}\ 10\ (\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3)) \longrightarrow \texttt{sub}\ 10\ (\texttt{add}\ 3\ (\texttt{add}\ 2\ 3))$ (we did this)

$\texttt{sub}\ 10\ (\texttt{add}\ 3\ (\texttt{add}\ 2\ 3)) \longrightarrow \texttt{sub}\ 10\ (\texttt{add}\ 3\ 5)$ (you did this)

$\texttt{sub}\ 10\ (\texttt{add}\ 3\ 5) \longrightarrow \texttt{sub}\ 10\ 8$ (exercise)

$\texttt{sub}\ 10\ 8 \longrightarrow 2$ (easy)

» Derive all necessary single-step evaluations

Traces

sub 10 (add (add 1 2) (add 2 3)) ⟶

sub 10 (add 3 (add 2 3)) ⟶

sub 10 (add 3 5) ⟶

sub 10 8 ⟶ 2

A **trace** is a sequence of (valid) single-step reductions

How To: Derivations of Multi-Step Reductions

$$\mathsf{sub}\ 10\ (\mathsf{add}\ (\mathsf{add}\ 1\ 2)\ (\mathsf{add}\ 2\ 3)) \longrightarrow^{\star} 2$$

» Derive all necessary single-step evaluations

» Combine them with the **transitivity rule**

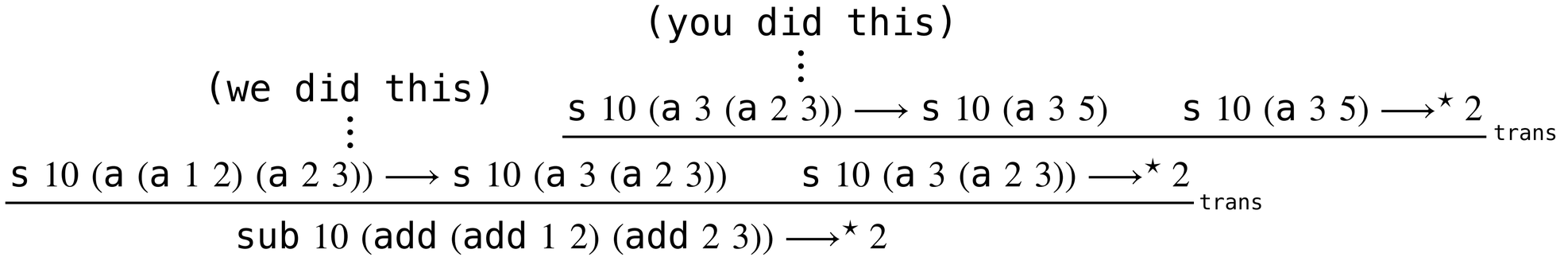
How To: Derivations of Multi-Step Reductions

$$\dfrac{\begin{array}{c}\texttt{(we did this)}\\ \vdots\\ \texttt{s}\ 10\ (\texttt{a}\ (\texttt{a}\ 1\ 2)\ (\texttt{a}\ 2\ 3)) \longrightarrow \texttt{s}\ 10\ (\texttt{a}\ 3\ (\texttt{a}\ 2\ 3)) \qquad \texttt{s}\ 10\ (\texttt{a}\ 3\ (\texttt{a}\ 2\ 3)) \longrightarrow^{\star} 2\end{array}}{\texttt{sub}\ 10\ (\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3)) \longrightarrow^{\star} 2}\ \texttt{trans}$$

» Derive all necessary single-step evaluations

» Combine them with the **transitivity rule**

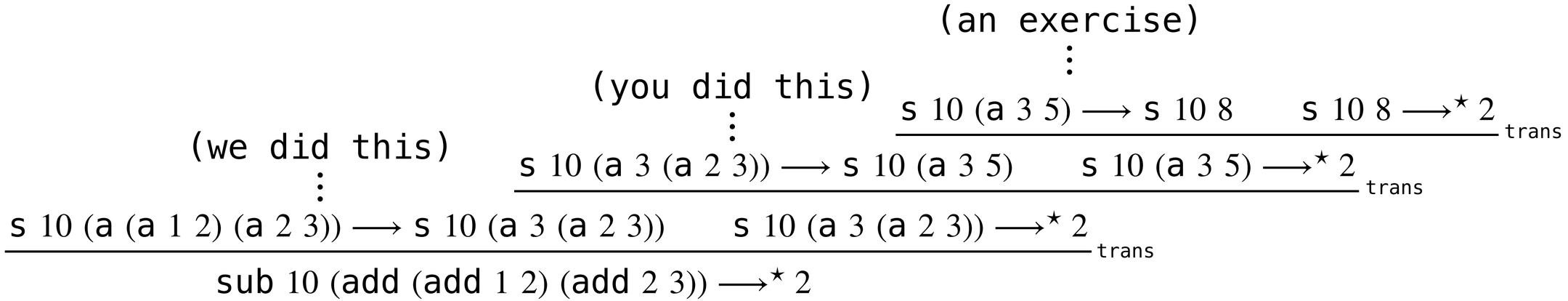
How To: Derivations of Multi-Step Reductions



» Derive all necessary single-step evaluations

» Combine them with the **transitivity rule**

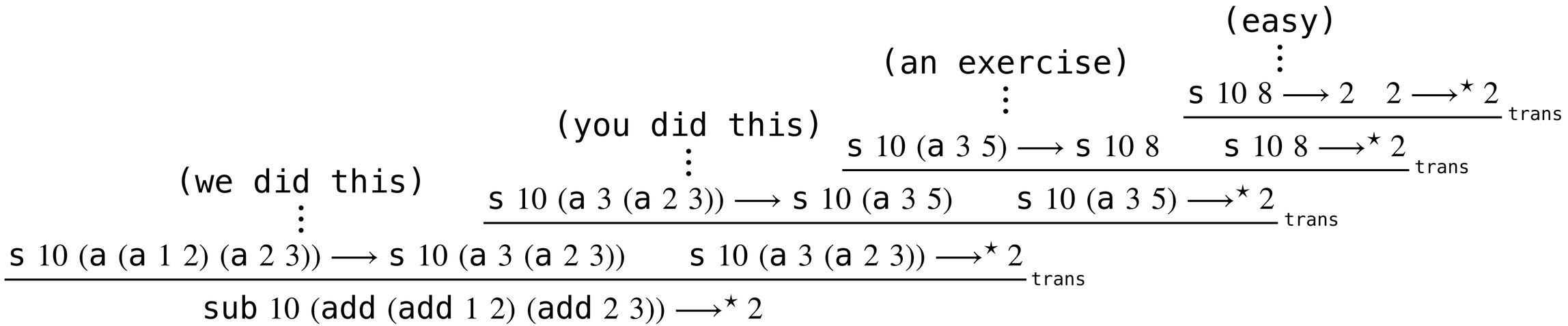
How To: Derivations of Multi-Step Reductions



» Derive all necessary single-step evaluations

» Combine them with the **transitivity rule**

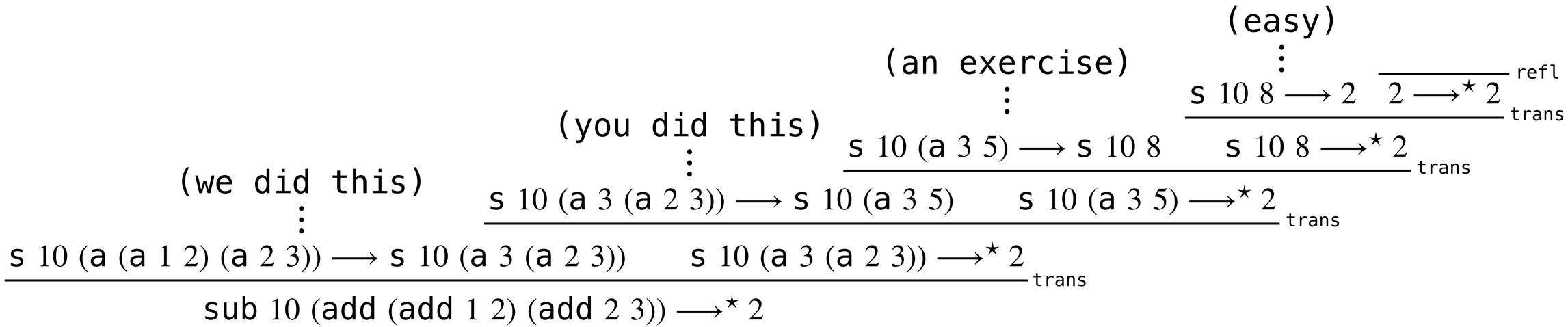
How To: Derivations of Multi-Step Reductions



» Derive all necessary single-step evaluations

» Combine them with the **transitivity rule**

How To: Derivations of Multi-Step Reductions



» Derive all necessary single-step evaluations

» Combine them with the **transitivity rule**

Two Questions (Again)

Once we have an operational semantics, there are two questions we can ask (as PL designers and on the final exam):

» Show that $C \longrightarrow^\star C'$

» Given C, determine a configuration C' such that $C \longrightarrow^\star C'$ **and** C' **cannot be reduced**

Two Questions (Again)

Once we have an operational semantics, there are two questions we can ask (as PL designers and on the final exam):

» Show that $C \longrightarrow^\star C'$

» Given C, determine a configuration C' such that $C \longrightarrow^\star C'$ **and** C' **cannot be reduced**

How To: Evaluation

$\texttt{sub}\ 10\ (\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3)) \longrightarrow^{\star} ??$

want to show

$\texttt{sub}\ 10\ (\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3)) \longrightarrow ??$

Solve this single-step evaluation problem until you reach a configuration that cannot be further reduced

How To: Evaluation

$$\text{sub } 10 \text{ (add (add } 1 \text{ } 2) \text{ (add } 2 \text{ } 3)) \longrightarrow^{\star} \text{??}$$

want to show

$$\text{sub } 10 \text{ (add (add } 1 \text{ } 2) \text{ (add } 2 \text{ } 3)) \longrightarrow \text{sub } 10 \text{ (add } 3 \text{ (add } 2 \text{ } 3))$$

$$\text{sub } 10 \text{ (add } 3 \text{ (add } 2 \text{ } 3)) \longrightarrow \text{??}$$

Solve this single-step evaluation problem until you reach a configuration that cannot be further reduced

How To: Evaluation

$$\texttt{sub}\ 10\ (\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3)) \longrightarrow^{\star} ??$$

want to show

$$\texttt{sub}\ 10\ (\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3)) \longrightarrow \texttt{sub}\ 10\ (\texttt{add}\ 3\ (\texttt{add}\ 2\ 3))$$

$$\texttt{sub}\ 10\ (\texttt{add}\ 3\ (\texttt{add}\ 2\ 3)) \longrightarrow \texttt{sub}\ 10\ (\texttt{add}\ 3\ 5)$$

$$\texttt{sub}\ 10\ (\texttt{add}\ 3\ 5) \longrightarrow ??$$

Solve this single-step evaluation problem until you reach a configuration that cannot be further reduced

How To: Evaluation

$$\texttt{sub}\ 10\ (\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3)) \longrightarrow^{\star} ??$$

want to show

$$\texttt{sub}\ 10\ (\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3)) \longrightarrow \texttt{sub}\ 10\ (\texttt{add}\ 3\ (\texttt{add}\ 2\ 3))$$

$$\texttt{sub}\ 10\ (\texttt{add}\ 3\ (\texttt{add}\ 2\ 3)) \longrightarrow \texttt{sub}\ 10\ (\texttt{add}\ 3\ 5)$$

$$\texttt{sub}\ 10\ (\texttt{add}\ 3\ 5) \longrightarrow \texttt{sub}\ 10\ 8$$

$$\texttt{sub}\ 10\ 8 \longrightarrow ??$$

Solve this single-step evaluation problem until you reach a configuration that cannot be further reduced

How To: Evaluation

$$\text{sub } 10 \text{ (add (add } 1\ 2) \text{ (add } 2\ 3)) \longrightarrow^{\star} 2$$

want to show

$$\text{sub } 10 \text{ (add (add } 1\ 2) \text{ (add } 2\ 3)) \longrightarrow \text{sub } 10 \text{ (add } 3 \text{ (add } 2\ 3))$$

$$\text{sub } 10 \text{ (add } 3 \text{ (add } 2\ 3)) \longrightarrow \text{sub } 10 \text{ (add } 3\ 5)$$

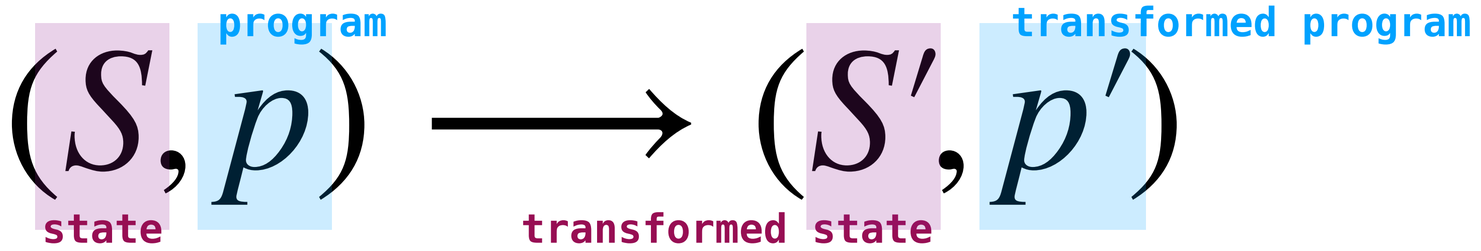
$$\text{sub } 10 \text{ (add } 3\ 5) \longrightarrow \text{sub } 10\ 8$$

$$\text{sub } 10\ 8 \longrightarrow 2$$

Solve this single-step evaluation problem until you reach a configuration that cannot be further reduced

When are we "done" computing?

Recall: Small-Step Semantics



Defining a small-step semantics means three things:

» What kind of **state**?

» What **rules** for transforming configurations?

» What are the **successful** configurations?

Irreducible Configurations/Normal Forms

An **irreducible configuration** is one that cannot be further reduced

Not all irreducible configurations are "successes"

Final Configurations/Values

We need to say which configurations are "success"

We often do this by specifying a subset of the (abstract) syntax

An irreducible configuration that is not final is called **stuck**

Three End-States

Three End-States

» **value:** we reach the value of our program

Three End-States

» **value:** we reach the value of our program

» **stuck:** we reach an expression that cannot be reduced, but that is not a value

```
(fun x -> x) (2 + 3) →* 5
```

Three End-States

» **value:** we reach the value of our program

`(fun x -> x) (2 + 3)` $\rightarrow^\star$ `5`

» **stuck:** we reach an expression that cannot be reduced, but that is not a value

`y (fun x -> x)` $\not\rightarrow$

» **diverge:** the computation never reaches a point where the expression is not reducible

`(fun x -> x x) (fun x -> x x)` $\rightarrow^\star$
`(fun x -> x x) (fun x -> x x)`

Big-Step Semantics

Big-Step Semantics

$$(\texttt{sub}\ 10\ (\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3))) \Downarrow 2$$

Big-Step Semantics

$$(\texttt{sub}\ 10\ (\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3))) \Downarrow 2$$

Big-step semantics deals only with a configuration and its value

Big-Step Semantics

$$(\texttt{sub}\ 10\ (\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3))) \Downarrow 2$$

Big-step semantics deals only with a configuration and its value

We write $e \Downarrow v$ to mean that e evaluates to the value v

Big-Step Semantics

$$(\texttt{sub}\ 10\ (\texttt{add}\ (\texttt{add}\ 1\ 2)\ (\texttt{add}\ 2\ 3))) \Downarrow 2$$

Big-step semantics deals only with a configuration and its value

We write $e \Downarrow v$ to mean that e evaluates to the value v

This is what we've been doing in this course so far

Example

```
<expr> ::= ( <op> <expr> <expr> )
         | <bool> | <int>
<op>   ::= add | sub | eq
<bool> ::= true | false
<int>  ::= ...
```

$$\frac{n \text{ is a number}}{n \Downarrow n}\ \text{numEval}$$

$$\frac{e_1 \Downarrow v_1 \qquad e_2 \Downarrow v_2 \qquad v_1 \text{ is a number} \qquad v_2 \text{ is a number}}{(\text{add } e_1\ e_2) \Downarrow v_1 + v_2}\ \text{addEval}$$

$$\frac{e_1 \Downarrow v_1 \qquad e_2 \Downarrow v_2 \qquad v_1 \text{ is a number} \qquad v_2 \text{ is a number}}{(\text{sub } e_1\ e_2) \Downarrow v_1 - v_2}\ \text{subEval}$$

Example

```
<expr> ::= ( <op> <expr> <expr> )
         | <bool> | <int>
<op>   ::= add | sub | eq
<bool> ::= true | false
<int>  ::= ...
```

$$\frac{n \text{ is a number}}{n \Downarrow n}\ \text{numEval}$$

$$\frac{e_1 \Downarrow v_1 \qquad e_2 \Downarrow v_2 \qquad v_1 \text{ is a number} \qquad v_2 \text{ is a number}}{(\text{add } e_1\ e_2) \Downarrow v_1 + v_2}\ \text{addEval}$$

$$\frac{e_1 \Downarrow v_1 \qquad e_2 \Downarrow v_2 \qquad v_1 \text{ is a number} \qquad v_2 \text{ is a number}}{(\text{sub } e_1\ e_2) \Downarrow v_1 - v_2}\ \text{subEval}$$

we'll remove these side conditions once we have type-checking

Practice Problem

```
<expr> ::= ( <op> <expr> <expr> )
         | <bool> | <int>
<op>   ::= add | sub | eq
<bool> ::= true | false
<int>  ::= ...
```

Write the rule for ***eq***

Answer

```
<expr> ::= ( <op> <expr> <expr> )
         | <bool> | <int>
<op>   ::= add | sub | eq
<bool> ::= true | false
<int>  ::= ...
```

Relation to Small-Step

$$e \longrightarrow^{\star} v \qquad \approx \qquad e \Downarrow v$$

Relation to Small-Step

$$e \longrightarrow^{\star} v \qquad \approx \qquad e \Downarrow v$$

The big-step relation **"cuts out the middle steps"** of a small-step relation

Relation to Small-Step

$$e \longrightarrow^{\star} v \qquad \approx \qquad e \Downarrow v$$

The big-step relation **"cuts out the middle steps"** of a small-step relation

This means fewer and clearer rules, but less fine-grain control of the evaluation sequence

Relation to Small-Step

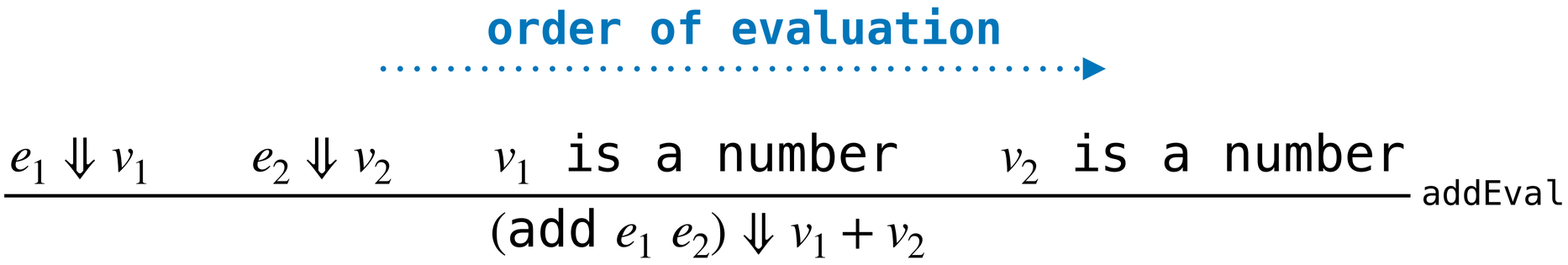
$$e \longrightarrow^{\star} v \qquad \approx \qquad e \Downarrow v$$

The big-step relation **"cuts out the middle steps"** of a small-step relation

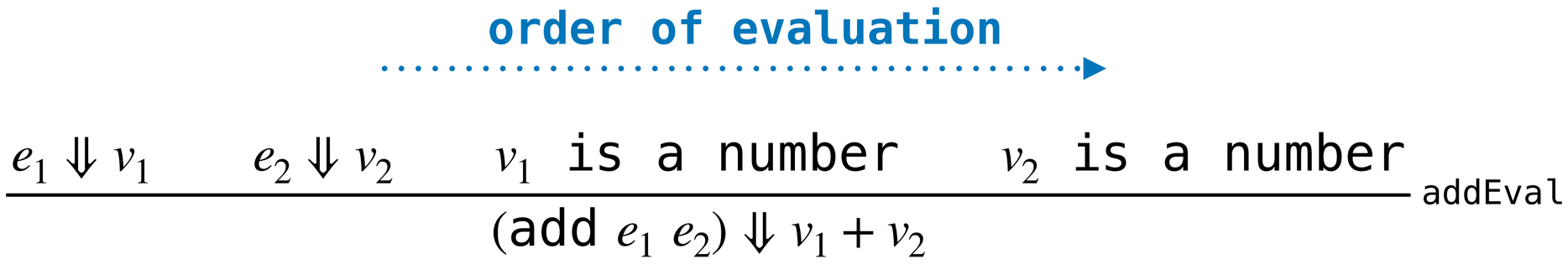
This means fewer and clearer rules, but less fine-grain control of the evaluation sequence

Note: We can't always have both small-step and big-step!

Order of Evaluation

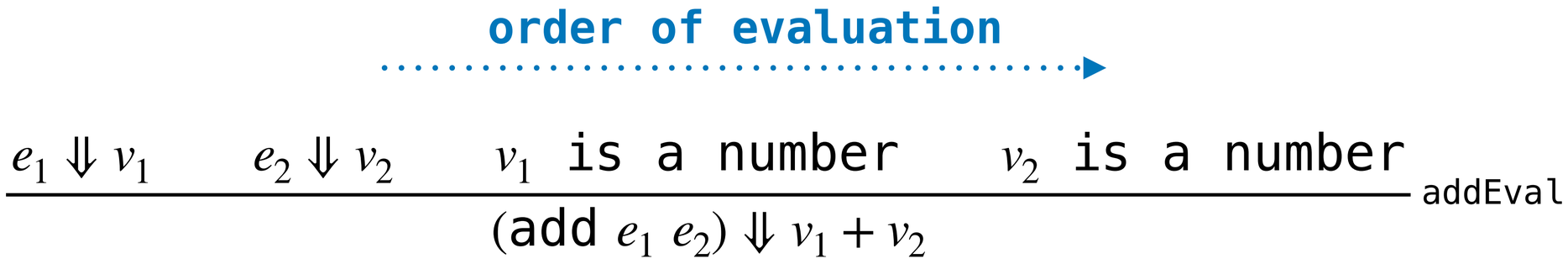


Order of Evaluation



With small-step semantics, we can choose the order of evaluations based on the rules

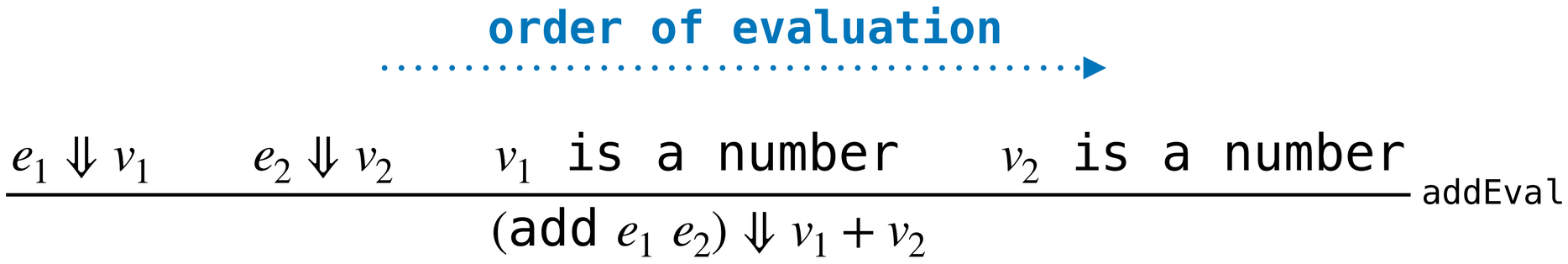
Order of Evaluation



With small-step semantics, we can choose the order of evaluations based on the rules

With big-step semantics, we can't because our relation only deals with the *final* value, nothing intermediate

Order of Evaluation



With small-step semantics, we can choose the order of evaluations based on the rules

With big-step semantics, we can't because our relation only deals with the *final* value, nothing intermediate

We will take the order of evaluation to be from left to right

Summary

big-step

$$e \Downarrow v$$

e evaluates to v

single-step

$$e \longrightarrow e'$$

e reduces to e' in a single step

multi-step

$$e \longrightarrow^{\star} e'$$

e reduces to e' in many steps